

DPDP COMPLIANCE AUDIT

finance-data-processing

Digital Personal Data Protection Act 2023



Grade B — Adequate

Scanned: 14 April 2026

Files indexed: 15

Languages: express, pg, jsonwebtoken, bcryptjs, express-validator, cors

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

Quick Actions for Developers

Fix these first to improve your compliance score:

- **GRIEVANCE_OFFICER_MISSING** (codebase-wide)
- **PASSWORD_NOT_HASHED** → `src/config/schema.sql`
- **AUDIT_TRAIL_PARTIAL** (codebase-wide)

This codebase has 1 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 2 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 1.0–3.6 engineering days.

The application collects user names, emails, and passwords, as well as financial record details. The primary risk lies in the potential for unauthorized access to sensitive financial data and user credentials due to the lack of explicit consent mechanisms for data collection and processing.

Total findings: 11

- Rule Engine: 11
- AI Deep Review: 0

By Severity:

- HIGH: 1
- MEDIUM: 2
- LOW: 2
- INFO: 3
- PASS: 3

Developer Priority:

- Fix Now: 1

Tech Stack: express, pg, jsonwebtoken, bcryptjs, express-validator

Risk Profile: The application collects user names, emails, and passwords, as well as financial record details. The primary risk lies in the potential for unauthorized access to sensitive financial data and user credentials due to the lack of explicit consent mechanisms for data collection and processing.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	22/22 (100%)	PASS
Section 6(6)	8	8/8 (100%)	PASS
Section 7	4	4/4 (100%)	PASS
Section 8(1)	18	9.0/18 (50%)	WARN
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	8/8 (100%)	PASS
Section 8(4)	8	4.0/8 (50%)	WARN
Section 8(6)	8	6.8/8 (85%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS

Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal	0	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 1.0–3.6 engineering days

Scan Coverage

Files analyzed: 15

Rule	Section	Severity	File
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
PASSWORD_NOT_HASHED	Section 8(1) — Security	MEDIUM	src/config/schema.sql
AUDIT_TRAIL_PARTIAL	Section 8(4) — Audit Trail	MEDIUM	N/A
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_WITHDRAWAL_PRESENT	Section 6(6) — Consent Withdrawal	PASS	src/controllers/authController.js
DELETION_MECHANISM_PRESENT	Section 8 — Data Principal Rights	PASS	src/routes/records.js
DATA_ACCESS_PRESENT	Section 11 — Right to Access	PASS	src/routes/auth.js

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (1–4 hours)

Fix immediately — these represent the highest legal exposure.

- Grievance Officer Missing (1–4 hrs)

Week 2 — High Priority (3–11 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Password Not Hashed — config/schema.sql (1–3 hrs)
- Audit Trail Partial (2–8 hrs)

Week 4 — Advisory (4–14 hours)

Address before next compliance review.

- No Breach Alerting (2–6 hrs)
- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 8–29 hours (1.0–3.6 engineering days)

Changes Since 2026-04-08

▼ **Score: 88.0 → 83 (-5.0 points)**

- ✓ 3 finding(s) resolved
- 1 new finding(s) introduced
- 10 finding(s) unchanged

Resolved since last scan:

- ✓ Incomplete Deletion Coverage
- ✓ Internal Route Pii Access
- ✓ Consent Present

New findings this scan:

- Password Not Hashed [HIGH]

Details

Finding 1 of 11 — ■ FIX NOW

[Section 13 — Grievance Redressal]

1. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A

Estimated fix time: 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route ****and**** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The absence of a designated Grievance Officer or DPO means individuals have no clear channel to address concerns or exercise their data protection rights, potentially leading to unresolved issues and non-compliance with DPDP Section 13.

Fix:

1. Create a new route (e.g., `/api/grievance`) that provides information about the grievance redressal mechanism.
2. In the new grievance route, clearly state the role and responsibilities of the Grievance Officer/DPO.
3. Provide a dedicated contact email address (e.g., `grievance@yourcompany.com` or `dpo@yourcompany.com`) for individuals to submit their concerns.
4. Ensure this route is accessible and clearly linked from the application's privacy policy or contact information page.

DPDP:

Section 13(1) — Grievance Redressal

Example:

```
const express = require('express'); const router = express.Router(); router.get('/grievance', (req, res) => { res.json({ message: 'Grievance Redressal Information', details: 'For any data protection concerns, please contact our Grievance Officer at grievance@example.com.', contactEmail: 'grievance@example.com' }); }); module.exports = router;
```

Finding 2 of 11 — ■ REVIEW & FIX

[Section 8(1) — Security]

[NEW]**2. [MEDIUM] PASSWORD_NOT_HASHED** — src / config / schema.sql:41

Evidence snippet: The line ``const hashed = await bcrypt.hash(password, 12);`` shows that the password is being hashed using bcrypt before being stored.

Estimated fix time: 1–3 hours

Model file has password field but no hash/bcrypt/argon/pbkdf2/scrypt/encrypt reference.

Risk:

The rule engine incorrectly flagged this finding. The code explicitly uses ``bcrypt.hash`` and ``bcrypt.compare`` to handle password security, indicating that passwords are being hashed before storage and verified correctly during login.

Fix:

1. No fix is required as the password hashing mechanism is already implemented correctly using bcrypt.

DPDP:

Section 8(1) — Security

Example:

```
const bcrypt = require('bcryptjs'); // Example of hashing a password async function
hashPassword(password) { const saltRounds = 12; return await bcrypt.hash(password,
saltRounds); } // Example of comparing a password async function
comparePassword(plainPassword, hashedPassword) { return await bcrypt.compare(plainPassword,
hashedPassword); }
```

Finding 3 of 11 — ■ REVIEW & FIX

[Section 8(4) — Audit Trail]

3. [MEDIUM] AUDIT_TRAIL_PARTIAL — N / A:13

Evidence snippet: The ``validate`` middleware is used, which might include some form of input validation that could be part of an audit trail.

Estimated fix time: 2–8 hours

Partial audit trail detected. Found: change tracking. Missing: access logging, audit model.

Risk:

While some change tracking (like password hashing) is present, the lack of comprehensive access logging and a dedicated audit model means there's insufficient visibility into who accessed what data and when, hindering investigations and compliance.

Fix:

1. Implement middleware to log all incoming API requests, including user ID (if authenticated), timestamp, HTTP method, and endpoint.
2. Create a dedicated ``audit_logs`` table in the database to store these access events, including fields like ``user_id``, ``action``, ``target_resource``, ``timestamp``, and ``ip_address``.
3. Modify controllers for sensitive operations (e.g., user creation, deletion, data modification) to log specific actions taken.
4. Integrate a logging library (e.g., Winston, Pino) to manage log rotation and storage effectively.

DPDP:

Section 8(4) — Audit Trail

Example:

```
// Example middleware for access logging
const pool = require('../config/database'); // Assuming you have a DB pool
const logAccess = async (req, res, next) => { const userId = req.user ? req.user.userId : null; await pool.query( 'INSERT INTO audit_logs (user_id, action, target_resource, ip_address) VALUES ($1, $2, $3, $4)', [userId, req.method + ' ' + req.originalUrl, req.params.id || 'N/A', req.ip] ); next(); }; // Apply this middleware to routes that need auditing, e.g., router.use(logAccess);
```

Finding 4 of 11 — ■ BACKLOG

[Section 8(6) — Breach]

4. [LOW] NO_BREACH_ALERTING — N / A**Estimated fix time:** 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

The absence of a breach notification mechanism means that in the event of a data breach, affected individuals and the Data Protection Authority will not be promptly informed, leading to potential regulatory penalties and loss of trust.

Fix:

1. Integrate a third-party service (e.g., Sentry, PagerDuty, Opsgenie) for real-time error and security event monitoring and alerting.
2. Configure the chosen service to trigger alerts for critical security events, such as unauthorized access attempts or data exfiltration patterns.
3. Establish an escalation policy within the alerting service to notify a designated compliance team or individual responsible for breach response.
4. Develop a documented incident response plan that outlines the steps to be taken upon receiving a breach alert, including communication protocols.

DPDP:

Section 8(6) — Breach

Example:

```
// Example using a hypothetical alerting service
const AlertingService = require('./alertingService'); // Your custom module
async function handlePotentialBreach(eventDetails) {
  try {
    await AlertingService.triggerHighSeverityAlert(
      'Potential Data Breach Detected', eventDetails );
    console.log('Breach alert triggered and escalated.');
```

```
  } catch (error) {
    console.error('Failed to trigger breach alert:', error);
  }
}
// Call handlePotentialBreach() when a suspicious event is detected.
```

Finding 5 of 11 — ■ BACKLOG

[Section 11 — Data Portability]

5. [LOW] DATA_PORTABILITY_MISSING — N / A**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The application fails to provide users with a mechanism to export their personal data, which is a requirement under DPDP Section 11, potentially leading to non-compliance and user dissatisfaction.

Fix:

1. Create a new route (e.g., `/api/export/data`) that handles GET requests.
2. Implement middleware to authenticate and authorize the user requesting their data.
3. In the controller, fetch the authenticated user's personal data (name, email, financial records) from the database.
4. Format the data into a machine-readable format (e.g., JSON or CSV) and send it as a file download response.

DPDP:

Section 11(1) — Data Portability requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Example:

```
const express = require('express'); const router = express.Router(); const { authenticate } = require('../middleware/auth'); const { getUserData } = require('../controllers/exportController'); // Assume this controller exists // Route to export user data router.get('/data', authenticate, getUserData); module.exports = router; // In exportController.js: // const getUserData = async (req, res) => { // const userId = req.user.id; // // Fetch user data and related financial records from DB // const data = await fetchUserDataFromDB(userId); // res.setHeader('Content-disposition', 'attachment; filename=user_data.json'); // res.json(data); // };
```

Finding 6 of 11 — ■ BACKLOG

[Section 5 — Notice]

6. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 7 of 11 — ■ BACKLOG

[Section 5 — Purpose Limitation]

7. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 8 of 11 — ■ BACKLOG

[Section 16 — Cross-Border Transfer]

8. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 9 of 11 — ■ BACKLOG

[Section 6(6) — Consent Withdrawal]

9. [PASS] CONSENT_WITHDRAWAL_PRESENT — src / controllers / authController.js

Consent withdrawal mechanism detected. Found in: src/controllers/authController.js (line 74)

Finding 10 of 11 — ■ BACKLOG

[Section 8 — Data Principal Rights]

10. [PASS] DELETION_MECHANISM_PRESENT — src / routes / records.js

Data deletion mechanism detected. Found 2 deletion endpoint(s) and 7 deletion signal(s). Note: Verify deletion cascades to all PII tables (sessions, logs, related records).

Fix:

1. Verify deletion cascades to: sessions, audit_logs, third-party sync records.
2. Use database CASCADE DELETE or explicit multi-table deletion in deletion service.
3. Test deletion completeness: create account, delete, query all PII tables for orphaned data.

Finding 11 of 11 — ■ BACKLOG

[Section 11 — Right to Access]

11. [PASS] DATA_ACCESS_PRESENT — src / routes / auth.js

Data access endpoint detected for authenticated users.